```
// The visitor-facing side: two MCP surfaces, one links page where every
// section is an installed app, and availability that captures the search
// before anyone leaves.
//   node demo-surfaces.js
import fs from 'node:fs';
import { boot, contextFor } from './src/platform.js';
import * as db from './src/db.js';
import { install } from './src/kernel/installer.js';
import { request } from './src/kernel/executor.js';
import { grant } from './src/kernel/policy.js';
import { drain } from './src/kernel/events.js';
import { publicSurface, privateSurface, issueToken } from './src/kernel/mcp.js';
import { renderLinks, renderEmbed } from './src/kernel/links.js';

const line = (s) => console.log('\n' + s + '\n' + '-'.repeat(s.length));

await boot();

const biz = await db.one(
  `insert into business (slug, display_name) values ('circle-boats','Beachside Ci
const owner = await db.one(
  `insert into person (email, display_name) values ('matt@example.com','Matt') re
const staff = await db.one(
  `insert into person (email, display_name) values ('dana@example.com','Dana') re
const ownerM = await db.one(
  `insert into membership (person_id, business_id, role) values ($1,$2,'owner') r
const staffM = await db.one(
  `insert into membership (person_id, business_id, role) values ($1,$2,'staff') r

for (const cap of ['business.set_fact','business.set_hours','availability.define'
  await grant({ businessId: biz.id, role: 'owner', capability: cap, disposition:
}
await grant({ businessId: biz.id, role: 'staff', capability: 'availability.search
await grant({ businessId: biz.id, role: 'staff', capability: 'availability.hold',

const ctx = await contextFor({ personId: owner.id, businessSlug: 'circle-boats' }
for (const key of ['business_profile', 'availability', 'qr_menu']) {
  await install({ businessId: biz.id, packageKey: key, grantTo: ['owner'] });
}
await grant({ businessId: biz.id, role: 'owner', capability: 'qr_menu.add_item',

await request({ ctx, capability: 'business.set_fact', input: { key: 'profile.cate
await request({ ctx, capability: 'business.set_fact', input: { key: 'contact.phon
await request({ ctx, capability: 'business.set_hours',
```

```
  input: { days: [0,1,2,3,4,5,6].map(d => ({ weekday: d, opens: '09:00', closes:

for (const item of [
  { name: 'Single Seater', description: 'Solo cruiser, max 300 lbs', price: 150,
  { name: 'Double Seater', description: 'Extra-wide seats, max 450 lbs', price: 2
  { name: 'Mini Dock', description: 'Platform for coolers and gear', price: 25, c
]) await request({ ctx, capability: 'qr_menu.add_item', input: item });

for (const day of ['2026-08-24', '2026-08-25', '2026-08-26']) {
  await request({ ctx, capability: 'availability.define',
    input: { resource: 'Single Seater', date: day, starts: '09:00', ends: '13:00'
  await request({ ctx, capability: 'availability.define',
    input: { resource: 'Double Seater', date: day, starts: '14:00', ends: '18:00'
}
await drain();

// --- two surfaces ------------------------------------------------------
line('SURFACE ONE: PUBLIC, NO CREDENTIAL');
const pub = await publicSurface('circle-boats');
console.log(` ${pub.name}  (${pub.surface})`);
for (const r of pub.resources) console.log(`   ${r.uri.padEnd(38)} from ${r.provi
console.log(` tools: ${pub.tools.map(t => t.name).join(', ')}`);

line('SURFACE TWO: PRIVATE, TOKEN BOUND TO A MEMBERSHIP');
const noAuth = await privateSurface({ slug: 'circle-boats', token: null });
console.log(` without a token: ${noAuth.error}`);

const ownerToken = await issueToken({ businessId: biz.id, membershipId: ownerM.id
const staffToken = await issueToken({ businessId: biz.id, membershipId: staffM.id

const asOwner = await privateSurface({ slug: 'circle-boats', token: ownerToken })
const asStaff = await privateSurface({ slug: 'circle-boats', token: staffToken })
console.log(` owner token -> acting as ${asOwner.actingAs.role}, ${asOwner.tools.
console.log(` staff token -> acting as ${asStaff.actingAs.role}, ${asStaff.tools.
console.log(`   staff sees: ${asStaff.tools.map(t => t.capability + (t.dispositio
const scriptOnly = asOwner.tools.filter(t => t.scriptOnly).map(t => t.capability)
console.log(`   script-only on the owner surface: ${scriptOnly.join(', ') || 'non
console.log(' the same token cannot see another business. the same person, differ

// --- the links page ----------------------------------------------------
line('THE LINKS PAGE');
const html = await renderLinks('circle-boats');
fs.writeFileSync('/home/claude/ghost/links-preview.html', html);
const tiles = [...html.matchAll(/class="t">([^<]+)</g)].map(m => m[1]);
console.log(` sections rendered: ${tiles.join(' | ')}`);
console.log(` every one is a projection from an installed app, not a hand-written
console.log(` page size: ${(html.length / 1024).toFixed(1)} kb, zero outbound lin
```

```
// --- the search is the point ------------------------------------------------
line('A VISITOR PICKS A DATE');
const before = await db.q(`select count(*)::int as n from search_intent where bus
console.log(` intent rows before: ${before[0].n}`);

const systemCtx = { businessId: biz.id, business: biz, person: null,
                    membership: { id: null, role: 'system' }, system: true };

const hit = await request({ ctx: systemCtx, capability: 'availability.search',
  input: { date: '2026-08-25', partySize: 2, surface: 'links' } });
console.log(` searched 2026-08-25 for 2 -> ${hit.result.matched} matches`);
for (const s of hit.result.slots) console.log(`   ${s.resource.padEnd(15)} ${s.st

const miss = await request({ ctx: systemCtx, capability: 'availability.search',
  input: { date: '2026-08-30', partySize: 6, surface: 'links' } });
console.log(` searched 2026-08-30 for 6 -> ${miss.result.matched} matches`);

await request({ ctx, capability: 'availability.hold',
  input: { slotId: hit.result.slots[0].id, partySize: 2 } });

const intents = await db.q(
  `select surface, requested_for, party_size, matched, converted from search_inte
    where business_id = $1 order by created_at`, [biz.id]);
console.log('\n what the business now knows, which the booking platform would hav
for (const i of intents) {
  const on = i.requested_for instanceof Date ? i.requested_for.toISOString().slic
  console.log(`   ${i.surface.padEnd(7)} ${on}  party ${i.party_size ?? '-'}  ` +
              `${i.matched} matched  ${i.converted ? 'booked' : 'did not book'}`)
}
console.log(' the 6-person request on the 30th found nothing. that is a demand si

// --- the same section, on their own site ------------------------------------
line('THE SAME SECTION AS AN EMBED');
const embed = await renderEmbed('circle-boats', 'availability');
fs.writeFileSync('/home/claude/ghost/embed-preview.html', embed);
console.log(` /embed/circle-boats/availability -> ${(embed.length / 1024).toFixed
console.log(' same projection, dropped into the business\'s own website. one sour
```